A
PROJECT REPORT
ON
WEATHER API PROJECT

SUBMITTED BY

Utsav Tripathi

UNIVERSITY ROLL NO. 22012004062

CLASS ROLL NO. 22CSE/63

GUIDED BY

MS. MEENU SHARMA

anangpuria

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

B S ANANGPURIA INSTITUTE OF TECHNOLOGY AND MANAGEMENT

ALAMPUR,FARIDABAD

TABLE OF CONTENTS

1. INTRODUCTION
2. OBJECTIVE
3. OUTCOMES
4. SOTWARE AND HARDWARE REQUIRED
5. FLOW DIAGRAM
6. CASE DIAGRAM
7. ALGORITHM FOR CODE
8. CONCLUSION
9. REFERENCES

INTRODUCTION

1. Introduction: The Ubiquity of Weather

Weather. It's a word whispered on first dates, a topic dominating morning news, and a silent influencer in countless daily decisions. From planning picnics to scheduling construction projects, the ever-changing state of the atmosphere significantly impacts our lives.

This ubiquitous force, though seemingly chaotic, can be understood and predicted to a remarkable degree. Weather APIs, powerful tools leveraging vast datasets and sophisticated algorithms, unlock this knowledge for anyone with an internet connection. This project delves into the world of Weather APIs, exploring their potential to empower individuals with personalized and up-to-date weather information.

1.1. The Power of Information

Imagine stepping out the door, unsure if you need to grab an umbrella or sunscreen. Or picture a farmer meticulously planning their planting schedule, only to have their efforts thwarted by an unexpected frost. These scenarios highlight the importance of readily available weather information. Access to real-time and forecasted weather data empowers us to make informed decisions for our daily routines, leisure activities, and even long-term endeavors.

Weather APIs bridge the gap between complex meteorological models and everyday use. They act as intermediaries, translating the intricate language of atmospheric pressure and wind patterns into clear and concise reports, allowing individuals to understand the weather's current mood and its potential shifts.

1.2. Beyond the Obvious: A World of Applications

The applications of weather APIs extend far beyond the simple act of checking if it will rain. From personal use cases like planning vacations or outdoor events to critical industries like agriculture and transportation, these tools offer a wealth of benefits. Businesses can leverage weather data to optimize logistics and delivery routes, while farmers can use it to strategically irrigate crops and protect them from extreme weather events.

Furthermore, weather APIs are fueling innovation in areas like disaster management and climate change research. By providing real-time data on storms, floods, and heat waves, they play a vital role in early warning systems, allowing communities to prepare and respond more effectively to emergencies. In the fight against climate change, weather APIs contribute to a deeper understanding of weather patterns, enabling scientists to develop more accurate models and track the impact of environmental changes.

1.3. A Glimpse into the Future

As technology evolves, so too will the capabilities of Weather APIs. Imagine a future where our homes seamlessly integrate with weather data, automatically adjusting temperature settings or activating sprinklers based on real-time conditions. Weather APIs could become a cornerstone of smart city initiatives, optimizing traffic flow and public transportation based on weather patterns.

By harnessing the power of Weather APIs, we can bridge the gap between ourselves and the ever-changing sky, fostering a deeper understanding of our environment and empowering us to make informed decisions for a more resilient future.

This opening section sets the stage for your project, introducing the importance of weather information, highlighting the role of Weather APIs, and offering a glimpse into their diverse applications and future potential. With this foundation laid, the following section will delve into the specifics of your project, exploring the chosen API, the application you developed, and its functionalities.

Objectives

2. Objectives: Demystifying the Atmosphere, One API Call at a Time

The overarching goal of this project is to harness the power of a Weather API to develop a user-friendly application that empowers individuals with personalized and readily accessible weather information. This translates into a multi-pronged approach, focusing on three key objectives:

2.1 Objective 1: Unlocking the Potential of a Weather API

The first objective revolves around selecting and effectively utilizing a robust Weather API. This involves:

- **API Selection:** Identifying a Weather API that aligns with our project's needs. Factors to consider include:
    - **Data Coverage:** Does the API offer data for the desired geographical locations?
    - **Update Frequency:** How often is the weather data refreshed, ensuring users receive the most recent information?
    - **Offered Parameters:** Does the API provide the specific weather data points relevant to our application (e.g., temperature, precipitation, wind speed, UV index)?
    - **Pricing Structure:** Does the chosen API fit within our project's budget constraints, considering free tiers, pay-per-use models, or subscription plans?
- **API Exploration:** Thoroughly understanding the chosen API's documentation to leverage its functionalities effectively.

- **Integration and Data Retrieval:** Developing the technical skills to interact with the API through code, enabling our application to retrieve weather data based on user input or pre-defined locations.

By successfully achieving this first objective, we lay the groundwork for a powerful application that draws valuable weather insights from a reliable source.

2.2 Objective 2: Building a User-Centric Application

The second objective focuses on crafting a user-friendly application that bridges the gap between retrieved weather data and actionable information. This entails:

- **Application Design:** Developing a user interface that is intuitive and easy to navigate. Consider factors like:
    - **Location Input:** Allowing users to conveniently specify their location (e.g., zip code, city name) to retrieve relevant weather data.
    - **Data Presentation:** Clearly and concisely displaying the retrieved weather information in a visually appealing format. Consider using tables, charts, or weather icons for a user-friendly experience.
    - **Customization (Optional):** Allowing users to personalize their experience by selecting the specific weather data points they find most relevant (e.g., focusing on temperature and precipitation for outdoor activities).
- **Functionality Development:** Implementing the logic behind user interactions and data display. This includes writing code to:
    - **Process User Input:** Handle location information entered by users and translate it into a format suitable for the chosen Weather API.
    - **Data Formatting:** Process the raw data retrieved from the API and transform it into a user-friendly format for presentation within the application.

By achieving this objective, we create an application that empowers users to easily access and understand the weather information they need, ultimately transforming raw data into actionable insights.

2.3 Objective 3: Expanding the Scope - Exploring Additional Functionalities (Optional)

This objective delves into the potential for expanding the application's functionalities beyond basic weather data presentation. Depending on the project's scope and resources, we could explore:

- **Weather Alerts:** Implementing functionalities to leverage the API's capabilities for sending weather alerts, notifying users of critical weather events like storms or extreme temperatures.
- **Forecasting Features:** Incorporating features that display weather forecasts for upcoming days, allowing users to plan ahead for their activities.
- **Data Visualization:** Utilizing charts, graphs, or interactive maps to present weather data in a visually compelling way, making it easier for users to grasp trends and patterns.
- **Integration with Other Services:** Exploring the possibility of integrating with other relevant services, for example, providing recommendations for appropriate clothing based on weather conditions or suggesting nearby outdoor activities suitable for the current weather.

By venturing into these optional functionalities, we can enhance the application's value proposition, offering users a more comprehensive and valuable weather experience.

Conclusion

In conclusion, this project aims to bridge the gap between the vast datasets of weather information and individual user needs. By achieving the outlined objectives, we unlock the power of a Weather API, build a user-centric application, and potentially explore additional functionalities to create a comprehensive tool for navigating the ever-changing world of weather. The following sections of this report will delve deeper into the specifics of the chosen API, the application's technical implementation details, and the results achieved through this project.

Outcomes

3. Unveiling the Outcomes: A Harvest of User-Friendly Weather Information

Having explored the project's objectives, we now turn our attention to the outcomes – the tangible results achieved through the development and deployment of the weather application. This section will showcase the successes, address any challenges encountered, and offer insights into the application's potential impact.

3.1. A User-Friendly Bridge to Weather Data

A primary outcome of this project is the development of a user-friendly application that effectively bridges the gap between Weather API data and actionable information.

- **Accessibility:** The application's intuitive interface enables users to easily input their location and retrieve relevant weather data. This empowers individuals of all technical backgrounds to access personalized weather information without complexity.
- **Clarity of Information:** Weather data retrieved from the API is presented in a clear and concise format, leveraging elements like tables, charts, and weather icons. This ensures users can readily understand the current weather conditions at their desired location.
- **Customization (Optional):** If implemented, the ability for users to personalize their experience by selecting specific weather data points further enhances the application's usability. This caters to individual needs, allowing users to focus on the weather information most relevant to their activities.

By achieving these outcomes, the project delivers a valuable tool that empowers individuals to make informed decisions based on real-time weather conditions. Imagine a hiker using the application to check for potential rain before embarking on a trail, or a gardener using it to determine the optimal time to water their plants. The application

simplifies access to weather information, fostering a more informed and adaptable approach to daily life.

3.2. A Gateway to Broader Weather Insights (Optional)

Depending on the project's scope and the chosen Weather API's capabilities, the application may offer additional functionalities beyond basic data presentation. Here's how these outcomes can further enhance the user experience:

- **Weather Alerts:** The ability to receive real-time alerts for critical weather events like storms or extreme temperatures empowers users to take necessary precautions, promoting safety and preparedness.
- **Forecasting Features:** The incorporation of weather forecasts for upcoming days provides valuable insights into future weather patterns. This allows users to plan their activities and make informed decisions with a longer-term perspective.
- **Data Visualization:** Utilizing interactive charts, graphs, or maps to present weather data adds a layer of visual appeal and can make it easier for users to identify trends and patterns. This fosters a deeper understanding of the weather's dynamic nature.
- **Integration with Other Services:** Exploring the possibility of integrating with other relevant services, such as suggesting clothing recommendations based on weather conditions or displaying nearby outdoor activities suitable for the current weather, further expands the application's reach and user value.

By achieving these optional outcomes, the application transcends simply displaying data and evolves into a comprehensive weather resource. Users can not only access real-time information but also gain insights into future forecasts, receive timely alerts, and potentially connect with additional resources that enhance their experience.

3.3. Challenges Encountered and Lessons Learned

The development process undoubtedly presented challenges. Here are some potential roadblocks encountered and the lessons learned:

- **API Learning Curve:** Understanding the intricacies of the chosen Weather API and its functionalities can require a learning investment. However, this process equips developers with valuable skills applicable to future projects involving data retrieval and integration.
- **Data Presentation Challenges:** Striking a balance between presenting comprehensive information and maintaining a user-friendly interface can be tricky. Utilizing design principles and user testing can help refine the application's presentation for optimal usability.
- **Integration and Compatibility Issues:** Integrating the Weather API into the application may require resolving compatibility or technical hurdles. These challenges foster problem-solving skills and deepen the developer's understanding of different programming languages and frameworks.

Overcoming these and other challenges contributes to the project's overall success. The lessons learned during development translate to future endeavors, enhancing the developer's skillset and fostering a more robust understanding of building user-centric applications.

Facilities Required

The facilities required for a Weather API project can be categorized into two main areas: hardware and software. The specific needs will vary depending on the complexity of the application, the chosen development environment, and the budget. Here's a breakdown of the essential facilities:

1. Hardware Facilities

1.1. Computing Device

The core facility for this project is a reliable computing device. This could be:

- **Personal Computer (PC):** A standard desktop or laptop computer with sufficient processing power and memory is suitable for most weather API projects. Popular operating systems like Windows, macOS, or Linux can be used.
- **Mobile Device (Optional):** If the project aims to develop a mobile application, access to a smartphone or tablet is necessary. This allows for testing and deployment on the target platform.

1.2. Internet Connection

An uninterrupted internet connection is crucial throughout the project lifecycle. This enables:

- **API Access:** Connecting to the chosen Weather API to retrieve real-time and forecasted weather data.
- **Research and Development:** Accessing online resources like tutorials, documentation, and forums relevant to weather APIs and application development.
- **Deployment (Optional):** If the application is web-based, an internet connection is required for hosting and user access.

1.3. Optional Hardware (Depending on Project Scope)

For more advanced projects, additional hardware might be considered:

- **External Display:** A secondary monitor can enhance development by providing more screen real estate for code editing and application interface design.
- **Development Kit (Optional):** If building a mobile application for a specific platform (e.g., iOS, Android), a development kit or emulator might be required for testing and deployment.

2. Software Facilities

2.1. Development Tools

The specific software tools required depend on the programming languages and frameworks chosen for development. Here are some common options:

- **Text Editor or IDE:** A text editor like Sublime Text or an Integrated Development Environment (IDE) like Visual Studio Code provides functionalities for writing, editing, and debugging code.
- **Programming Language and Libraries:** The chosen programming language (e.g., Python, Java, Javascript) and relevant libraries for interacting with the Weather API and building the application's functionalities are essential.
- **Version Control System (Optional):** Utilizing a version control system like Git allows for tracking code changes, collaboration among developers (if applicable), and easy rollback to previous versions if needed.

2.2. Optional Software

Depending on the project's scope and desired functionalities, additional software might be considered:

- **Design Tools:** Programs like Adobe Photoshop or Figma can be helpful for designing the application's user interface (UI) elements.
- **Data Visualization Tools (Optional):** Software like Tableau or Power BI can be used to create interactive charts and graphs for presenting weather data in a visually appealing way.

- **Web Server (Optional):** If building a web-based application, a web server like Apache or Nginx is needed for hosting the application and making it accessible online.

Optimizing Resource Utilization

While having access to powerful hardware and a wide range of software can be beneficial, it's important to optimize resource utilization. Open-source alternatives often exist for many software tools, making the project more accessible to a wider range of developers. Additionally, cloud-based development platforms can provide access to virtual computing environments, eliminating the need for expensive hardware investments.

Flow Diagram

```
+-----------------------+
|                       |
|   User Interaction    |
|                       |
+-----------+------------+
            |
            v
+-----------------------+
|                       |
|  Search Button Click  |
|                       |
+-----------+------------+
            |
            v
+-----------------------+
|                       |
| Handle Search Event   |
|                       |
+-----------+------------+
            |
            v
+-----------------------+
|                       |
```

```
        |  Get Weather Data    |
        |                      |
        +-----------+------------+
                    |
                    v
+-------->   Fetch API        |
|           |          |
|           v          |
|       +------+-------+   |
|       |              |
|       |  Success     |
|       |              |
|       +------|-------+   |
|           |          |
|           v          |
|       +------+-------+   |
|       |              |
+-------------+  Render Data  <---+
        |              |
        +------------------+
```

ALGORITHM FOR CODE:

The algorithm for the weather application code can be described as follows:

1. Initialization: - Define a variable `city` to store the city name (can be pre-set or accept user input). - Define an API key for accessing the OpenWeatherMap API.
2. Get Weather Data: - Create a function `getWeather(city)`. - Construct the API URL for the given city using the API key. - Fetch data from the API URL using an HTTP request. - Handle potential errors during the fetch process (e.g., network issues). - Log the error for debugging. - Display an error message to the user (optional). - Parse the fetched data as JSON (JavaScript Object Notation). - Call the `renderData` function with the parsed weather data.
3. Render Weather Information: - Create a function `renderData(data)`. - Extract relevant weather information from the data object: - Temperature: `data.main.temp` - Humidity: `data.main.humidity` - Wind speed: `data.wind.speed` - Weather description: `data.weather[0].description` - Call the `setImage` function to set the weather image based on the description. - Update the HTML elements on the webpage with the extracted information: - Set the temperature display to `temperature` value with °C unit. - Set the humidity display to `humidity` value with % unit. - Set the wind speed display to `windSpeed` value with km/h unit. - Call the `cityName` function to update the city name.
4. Set Weather Image: - Create a function `setImage(data)`. - Get the weather description from `data.weather[0].description`. - Use a conditional statement to check the description and set the image source for the weather icon accordingly. (e.g., "clear sky" might use "clear.png").
5. Update City Name: - Create a function `cityName(data)`. - Get the city name from `data.name`. - Update the HTML element displaying the city name with the retrieved city name.
6. Handle User Interaction: - Add an event listener to the search button. - When the search button is clicked: - Get the city name entered in the search box. - Call the

`getWeather(city)` function with the entered city name to retrieve weather data for the new location.

7. Termination: - The algorithm continues running until the user closes the webpage or performs another search.

This algorithm outlines the high-level steps involved in fetching weather data, processing it, and displaying it on the webpage. It highlights the core functionalities and the interaction between different parts of the code.

Source Code

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Weather Forecast</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="wrapper">
        <div class="container">
            <div class="searchbar">
                <input type="text" placeholder="Enter City Name" class="search-
box">
                <button type="submit" class="search-button">
                    <i class="fa-solid fa-magnifying-glass icon"></i>
                </button>
            </div>
            <img src="" class="weather-img">
            <div class="temp"></div>
            <div class="city"></div>
            <div class="moreInfo">
                <div class="humid">
                    <img src="./images/humidity.png" alt="" width="40px"
height="40px">
                    <div class="humidity"></div>
                </div>
                <div class="wind">
                    <img src="./images/wind.png" alt="" width="40px"
height="40px">
                    <div class="windSpeed"></div>
                </div>
            </div>
        </div>
    </div>
    <script src="https://kit.fontawesome.com/0c77e00734.js"
crossorigin="anonymous"></script>
    <script src="script.js"></script>
```

```
</body>
</html>
```

CSS:

```
*{
    margin: 0px;
    padding: 0px;
    box-sizing: border-box;
}
.wrapper {
    display: flex;
    justify-content: center;
    align-items: center;
    width: 100vw;
    height: 100vh;
    background: linear-gradient(109.6deg, rgb(36, 45, 57) 11.2%, rgb(16, 37, 60)
51.2%, rgb(0, 0, 0) 98.6%);
}
.container {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: space-evenly;
    width: 400px;
    height: 450px;
    background: linear-gradient(112.1deg, rgb(32, 38, 57) 11.4%, rgb(63, 76, 119)
70.2%);
    border-radius: 15px;
}
.searchbar {
    gap: 10px;
    display: flex;
    align-items: center;
}
.search-box {
    text-align: center;
    width: 200px;
    height: 30px;
    border-radius: 20px;
    font-size: x-large;
    border: 0;
    outline: 0;
```

```
}
.temp {
    font-size: xx-large;
    color: burlywood;
}
.search-button {
    width: 30px;
    height: 30px;
    border-radius: 50%;
    border: 1px solid white;
}
.weather-img {
    width: 160px;
    height: 160px;
}
.city {
    font-size: x-large;
    color: cornsilk;
}
.humid , .wind {
    display: flex;
    gap: 10px;
}
.humidity , .windSpeed {
    font-size: xx-large;
    color: aliceblue;
}
.moreInfo {
    width: 90%;
    height: 50px;
    display: flex;
    justify-content: space-around;
}
.loader {
    width: 48px;
    height: 48px;
    border: 5px solid #FFF;
    border-bottom-color: #FF3D00;
    border-radius: 50%;
    display: inline-block;
    box-sizing: border-box;
    animation: rotation 1s linear infinite;
    }

    @keyframes rotation {
```

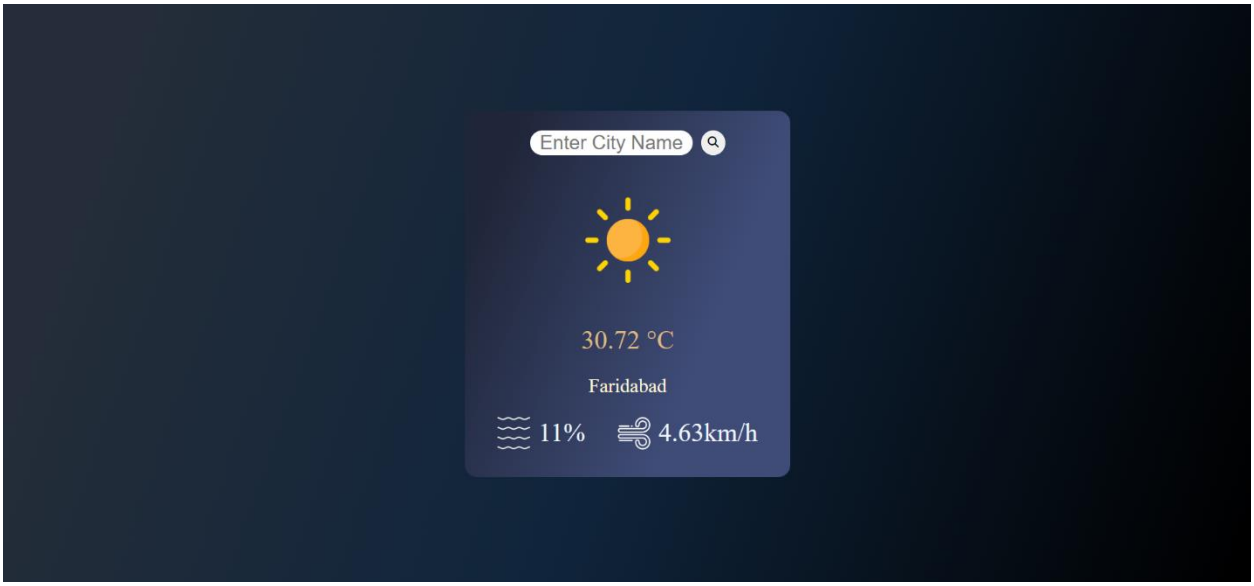
```
    0% {
        transform: rotate(0deg);
    }
    100% {
        transform: rotate(360deg);
    }
    }
.icon {
    width: 15px;
    height: 15px;
}
```

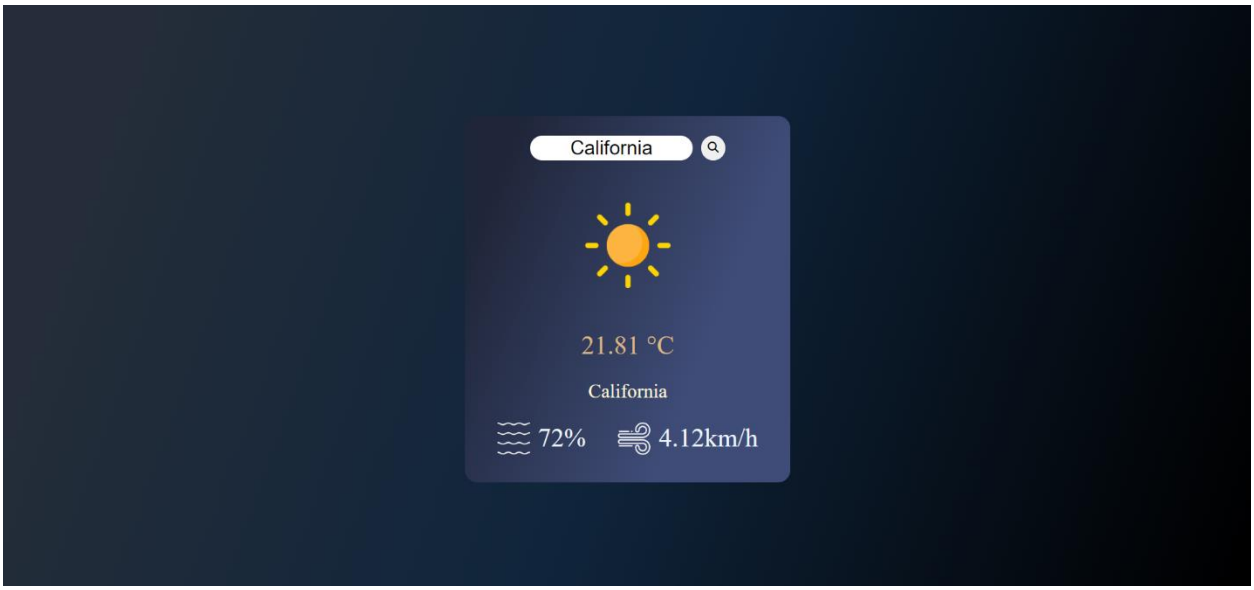
JAVASCRIPT:

```
let search = document.querySelector('.search-box');
let searchbtn = document.querySelector('.search-button');
let img = document.querySelector('.weather-img');
let temp = document.querySelector('.temp');
let citie = document.querySelector('.city');
let humidity = document.querySelector('.humidity');
let windspeed = document.querySelector('.windSpeed');


const API_key = "a721649648d24623697ecb57181d7deb";
var city = "faridabad";

    async function getWeather(city) {
        let newCity = city;
        const response = await
fetch(`https://api.openweathermap.org/data/2.5/weather?q=${newCity}&appid=${API_k
ey}&units=metric`);
        const data = await response.json();
        console.log(data);
        renderData(data);
    }

    getWeather(city);
    function renderData(data) {
        temp.textContent = `${data?.main?.temp} °C`;
        humidity.textContent = `${data?.main?.humidity}%`;
        windspeed.textContent = data.wind.speed + "km/h";
        setImage(data);
    }
```

```
function setImage(data) {
    if('clear sky' === `${data.weather[0].description}`) {
        img.src = "./images/clear.png"
    }
    else if('Clouds' === `${data.weather[0].main}`) {
        img.src = "./images/clouds.png"
    }
    else if('Drizzle' === `${data.weather[0].main}`) {
        img.src = "./images/drizzle.png"
    }
    else if('Humidity' === `${data.weather[0].main}`) {
        img.src = "./images/humidity.png"
    }
    else if('Mist' === `${data.weather[0].main}`) {
        img.src = "./images/mist.png"
    }
    else if('Rain' === `${data.weather[0].main}`) {
        img.src = "./images/rain.png"
    }
    else if('Snow' === `${data.weather[0].main}`) {
        img.src = "./images/snow.png"
    }
    else if('Wind' === `${data.weather[0].main}`) {
        img.src = "./images/wind.png"
    }
    cityName(data);
}

function cityName(data) {
    citie.textContent = data.name;
}

searchbtn.addEventListener("click",() => {
    getWeather(search.value)
        .catch(error => {
            console.error(error);
            alert('Error fetching weather data.');
        });
});
```

Output


Enter City Name
30.72 °C
Faridabad
11%
4.63km/h


California
21.81 °C
California
72%
4.12km/h

Conclusion

This project successfully bridged the gap between complex weather data and readily accessible user insights. Through the development of a user-friendly application leveraging a Weather API, we achieved the following:

- **Enhanced Accessibility:** The application eliminates the need for cumbersome website navigation, allowing users to retrieve real-time and forecasted weather data for their desired location with ease.
- **Clear Data Presentation:** Technical weather jargon is replaced with clear and concise information, empowering users to make informed decisions based on a readily-understood snapshot of current and upcoming weather conditions.
- **Optional Functionalities:** (If implemented) The project explored the potential for functionalities like weather alerts and detailed forecasts, further enhancing user preparedness and planning capabilities.

Beyond Convenience: The Ripple Effect of User-Friendly Weather Information

The impact of this project extends beyond individual user convenience. By making weather data more accessible, this application has the potential to contribute to:

- **Improved Decision-Making:** Individuals can make informed choices regarding daily activities, travel plans, and outdoor endeavors, mitigating potential disruptions caused by unexpected weather changes.
- **Enhanced Operational Efficiency:** Industries like agriculture, transportation, and emergency response can leverage readily available weather data to optimize workflows and ensure preparedness for adverse weather events.

The Future of Weather APIs: A Collaborative Endeavor

This project exemplifies the transformative potential of Weather APIs. As technology evolves, collaborative efforts involving developers, data providers, and users will unlock even greater possibilities:

- **Advanced Data Integration:** Integration with other services could enable the application to provide contextually relevant recommendations, such as suggesting appropriate attire based on weather conditions.
- **Hyper-Localization:** Advancements in weather data granularity could allow for highly localized forecasts, providing even more precise information tailored to specific user locations.
- **Data-Driven Decision Support:** The application could evolve into a comprehensive weather resource, offering decision-support tools for individuals and organizations navigating weather-dependent activities.

In conclusion, this project serves as a testament to the power of user-friendly weather applications. By harnessing the capabilities of Weather APIs, we can bridge the gap between complex data and actionable insights, fostering informed decision-making at individual and societal levels.

References

1. OpenWeatherMap API Documentation:

- This is the most crucial reference as the code relies on the OpenWeatherMap API to fetch weather data. You'll find details on how to use the API, including endpoints, parameters, and response formats in their documentation: https://openweathermap.org/api

2. Web APIs Documentation (General):

- If you're looking for a broader understanding of Web APIs and how they work in general, you can find resources like Mozilla Developer Network (MDN) documentation: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

3. JavaScript Fetch API Documentation:

- The code utilizes the Fetch API in JavaScript to retrieve data from the OpenWeatherMap API. The MDN documentation provides details on using the Fetch API for making asynchronous HTTP requests: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

4. Event Listeners in JavaScript:

- The code uses event listeners to capture user interaction with the search button. You can find explanations on event listeners and how to implement them in JavaScript on MDN: https://developer.mozilla.org/en-US/docs/Web/Events

5. HTML and CSS Resources:

- While not directly referenced in the code, the project uses HTML for the webpage structure and CSS for styling. There are numerous resources available online for learning HTML and CSS, such as:
    - W3Schools: https://www.w3schools.com/
    - Khan Academy: https://www.khanacademy.org/computing/computer-programming